



Relatório Final — Lab 05

Laboratório de Experimentação de Software
GraphQL vs REST: Um Estudo Experimental de Desempenho e Eficiência de Rede
Link do [Repositório](#)

Gustavo Delfino Guimarães
delfinoguimaraes1@gmail.com
Matrícula: 836079

Matheus Caetano Rocha
matheuscaetanorocha@gmail.com
Matrícula: 83839

Sumário

1. Introdução.....	3
1.1. Perguntas de Pesquisa.....	3
1.2. Hipóteses.....	3
2. Metodologia.....	3
2.1. Sistema Sob Teste.....	3
2.2. Desenho Experimental.....	4
2.3. Consultas Pareadas.....	4
2.4. Controle de Ameaças à Validade.....	4
2.5. Ambiente de Execução.....	5
2.6. Métodos Estatísticos.....	5
3. Resultados.....	5
3.1. Estatísticas Descritivas.....	5
3.2. Teste de Normalidade (Shapiro-Wilk).....	6
3.3. Testes de Hipótese (Wilcoxon Signed-Rank).....	6
3.4. Gráficos.....	7
4. Discussão.....	7
4.1. RQ1 — Tempo de Resposta.....	7
4.2. RQ2 — Tamanho da Resposta.....	8
4.3. Análise por Query.....	8
4.4. Limitações.....	8
5. Conclusão.....	8
Referências.....	9

1. Introdução

A linguagem de consulta **GraphQL**, proposta pelo Facebook em 2015, representa uma alternativa às populares APIs REST. Enquanto APIs REST se baseiam em endpoints pré-definidos que retornam estruturas fixas de dados, o GraphQL permite que o cliente especifique exatamente quais campos deseja receber, potencialmente reduzindo o tráfego de dados desnecessários (*overfetching*) e o número de requisições necessárias (*underfetching*).

Desde o seu surgimento, diversos sistemas de grande escala — incluindo o próprio GitHub — realizaram a migração para GraphQL, mantendo também suas APIs REST por compatibilidade. Entretanto, não está claro se a adoção de GraphQL traz benefícios mensuráveis em termos de **desempenho e eficiência de rede**.

1.1. Perguntas de Pesquisa

RQ	Pergunta
RQ1	Respostas às consultas GraphQL são mais rápidas que respostas REST?
RQ2	Respostas às consultas GraphQL têm tamanho menor que respostas REST?

1.2. Hipóteses

RQ1 — Tempo de Resposta

- H_{01} (Hipótese Nula): Não há diferença estatisticamente significativa no tempo de resposta entre consultas GraphQL e REST.
- H_{11} (Hipótese Alternativa): Há diferença estatisticamente significativa no tempo de resposta entre consultas GraphQL e REST.

RQ2 — Tamanho da Resposta

- H_{02} (Hipótese Nula): Não há diferença estatisticamente significativa no tamanho da resposta entre consultas GraphQL e REST.
- H_{12} (Hipótese Alternativa): Há diferença estatisticamente significativa no tamanho da resposta entre consultas GraphQL e REST.

2. Metodologia

2.1. Sistema Sob Teste

O experimento utiliza as duas APIs públicas do **GitHub**:

API	Base URL	Autenticação
REST API v3	https://api.github.com	Bearer token (PAT)
GraphQL API v4	https://api.github.com/graphql	Bearer token (PAT)

Ambas atendem ao mesmo domínio de dados (repositórios, usuários, issues, pull requests), tornando as consultas semanticamente equivalentes e viabilizando o design pareado.

2.2. Desenho Experimental

Aspecto	Definição
Variável Independente	Tipo de API: REST vs GraphQL
Variáveis Dependentes	Tempo de resposta (response_time_ms) e Tamanho (response_bytes)
Tratamentos	REST API v3 (GET) e GraphQL API v4 (POST)
Tipo de Projeto	Experimento pareado com randomização da ordem dos tratamentos
Objetos Experimentais	5 consultas pareadas (Q1–Q5) sobre os mesmos recursos
Repetições	30 iterações por consulta (após descarte de 3 iterações de warmup)
Total de medições	$30 \times 5 \times 2 = \mathbf{300 \text{ medições válidas}}$

2.3. Consultas Pareadas

ID	Label	REST Endpoint	GraphQL Query
Q1	user_profile	GET /users/octocat	query { user(login: ...) }
Q2	repository_metadata	GET /repos/octocat/Hello-World	query { repository(...) }
Q3	user_repositories_list	GET /users/octocat/repos?per_page=30	query { user { repos(...) } }
Q4	repository_issues_list	GET /repos/facebook/react/issues?..	query { repository { issues } }
Q5	repository_pull_requests	GET /repos/facebook/react/pulls?..	query { repository { PRs } }

As consultas GraphQL foram cuidadosamente construídas para solicitar **os mesmos campos** retornados por padrão pela API REST, garantindo equivalência semântica.

2.4. Controle de Ameaças à Validade

Ameaça	Mitigação
Cold start (DNS/TLS)	3 iterações de warmup descartadas antes da coleta de dados
Cache do servidor	Header Cache-Control: no-cache em todas as requisições
Efeito de ordenação	Ordem dos tratamentos (REST→GraphQL ou GraphQL→REST) randomizada a cada par
Variabilidade de rede	Sleep de 0.6s entre requisições para evitar burst; medições sob mesma conexão
Rate limiting	Monitoramento do X-RateLimit-Remaining ; abort automático se quota < 10%
Recurso alvo variável	Alvos fixos (octocat , facebook/react) em todas as iterações

2.5. Ambiente de Execução

Item	Valor
Linguagem	Python 3.10+
Bibliotecas	<code>requests</code> , <code>python-dotenv</code> , <code>pandas</code> , <code>scipy</code> , <code>matplotlib</code> , <code>seaborn</code>
Sistema Operacional	Windows
Medição de tempo	<code>time.perf_counter()</code> (alta resolução, monótono)
Medição de tamanho	<code>len(response.content)</code> — bytes brutos do corpo da resposta

2.6. Métodos Estatísticos

Análise	Método
Estatísticas descritivas	Média, mediana, desvio-padrão, IQR, min, max
Teste de normalidade	Shapiro–Wilk ($\alpha = 0.05$)
Teste de hipótese pareado	Wilcoxon signed-rank test (não paramétrico, $\alpha = 0.05$)
Tamanho de efeito	Cliff's $\tilde{D}$: negligível (< 0.147), pequeno (< 0.33), médio (< 0.474), grande (≥ 0.474)

A escolha do teste de **Wilcoxon** (não paramétrico) se justifica pela expectativa de distribuições não normais nos tempos de resposta de APIs web, que tipicamente apresentam cauda longa à direita.

3. Resultados

3.1. Estatísticas Descritivas

Visão Geral por Tratamento

Trat.	Métrica	N	Média	Mediana	Desv.Pad.	Min	Max	IQR
REST	<code>response_time_ms</code>	150	908,910	723,155	400,954	441,898	2.692,152	636,982
REST	<code>response_bytes</code>	150	140.772,0	41.788,0	194.180,5	1.232	514.432	136.026
GRAPHQL	<code>response_time_ms</code>	150	1.028,528	1.004,084	444,137	448,065	3.669,884	638,458
GRAPHQL	<code>response_bytes</code>	150	8.483,6	13.064,0	6.646,2	211	15.183	12.694

Visão por Query

Query	Tratamento	Tempo Médio (ms)	Tempo Mediana (ms)	Tamanho (bytes)
Q1	REST	634,778	521,809	1.232
Q1	GRAPHQL	567,216	535,372	211
Q2	REST	615,547	591,886	5.191

Query	Tratamento	Tempo Médio (ms)	Tempo Mediana (ms)	Tamanho (bytes)
Q2	GRAPHQL	672,677	657,188	633
Q3	REST	708,081	628,842	41.788
Q3	GRAPHQL	1.180,424	1.145,310	13.327
Q4	REST	1.154,686	1.146,106	141.217
Q4	GRAPHQL	1.160,809	1.056,198	13.064
Q5	REST	1.431,455	1.413,499	514.432
Q5	GRAPHQL	1.561,513	1.558,972	15.183

3.2. Teste de Normalidade (Shapiro-Wilk)

Tratamento	Métrica	Estatística W	p-valor	Normal ($\alpha=0,05$)?
REST	response_time_ms	0,882797	$\approx 0,000$	Não
REST	response_bytes	0,667425	$\approx 0,000$	Não
GRAPHQL	response_time_ms	0,881013	$\approx 0,000$	Não
GRAPHQL	response_bytes	0,699710	$\approx 0,000$	Não

Nenhuma das distribuições apresentou normalidade (todos os p-valores ≈ 0), justificando a escolha do teste não paramétrico de Wilcoxon para as análises subsequentes.

3.3. Testes de Hipótese (Wilcoxon Signed-Rank)

RQ1 — Tempo de Resposta

Escopo	N pares	Wilcoxon W	p-valor	Decisão H_{01}	Cliff's δ	Magnitude
Overall	150	3.091,0	0,000001	Rejeitada	-0,1714	small
Q1	30	214,0	0,715133	Não rejeitada	-0,1044	negligible
Q2	30	116,0	0,015460	Rejeitada	-0,4289	medium
Q3	30	2,0	$\approx 0,000$	Rejeitada	-0,9000	large
Q4	30	104,0	0,007111	Rejeitada	+0,3867	medium
Q5	30	80,0	0,001131	Rejeitada	-0,4844	large

Nota: Cliff's δ negativo indica que GraphQL tende a ser **mais lento** que REST. Cliff's δ positivo indica que REST tende a ser mais lento (ou seja, GraphQL é mais rápido).

RQ2 — Tamanho da Resposta

Escopo	N pares	Wilcoxon W	p-valor	Decisão H_0	Cliff's δ	Magnitud e
Overall	150	0,0	$\approx 0,000$	Rejeitada	+0,5200	large
Q1	30	0,0	$\approx 0,000$	Rejeitada	+1,0000	large
Q2	30	0,0	$\approx 0,000$	Rejeitada	+1,0000	large
Q3	30	0,0	$\approx 0,000$	Rejeitada	+1,0000	large
Q4	30	0,0	$\approx 0,000$	Rejeitada	+1,0000	large
Q5	30	0,0	$\approx 0,000$	Rejeitada	+1,0000	large

Nota: Cliff's $\delta = +1,0$ indica que REST retornou respostas maiores que GraphQL em **100% dos pares**, sem exceção.

3.4. Gráficos

Os seguintes gráficos foram gerados automaticamente pelo script `src/analysis.py` e estão disponíveis no diretório `output/`:

Gráfico	Arquivo
Boxplots gerais (tempo e tamanho)	<code>output/boxplots_overall.png</code>
Boxplots de tempo por query	<code>output/boxplots_time_per_query.png</code>
Boxplots de tamanho por query	<code>output/boxplots_size_per_query.png</code>
Barplot medianas de tempo por query	<code>output/barplot_time_medians.png</code>
Barplot medianas de tamanho por query	<code>output/barplot_size_medians.png</code>

4. Discussão

4.1. RQ1 — Tempo de Resposta

Os resultados indicam que, **no geral, GraphQL é levemente mais lento que REST** (mediana de 1.004 ms vs 723 ms), com diferença estatisticamente significativa ($p \approx 0,000001$) mas efeito de magnitude pequena (Cliff's $\delta = -0,17$).

Analisando por query, o comportamento é heterogêneo:

- **Q1 (user_profile):** Sem diferença significativa ($p = 0,715$). Consultas simples sobre um recurso único apresentam latência equivalente.
- **Q2 (repository_metadata):** GraphQL é significativamente mais lento ($\delta = -0,43$, médio). O overhead de parsing da query GraphQL supera o ganho em tamanho de resposta para este recurso.
- **Q3 (user_repositories_list):** GraphQL é substancialmente mais lento ($\delta = -0,90$, grande). A resolução de listas no GraphQL envolve processamento adicional no servidor.
- **Q4 (repository_issues_list):** REST é mais lento que GraphQL ($\delta = +0,39$, médio). Este é o único caso em que GraphQL apresentou vantagem em tempo.
- **Q5 (repository_pull_requests_list):** GraphQL é mais lento ($\delta = -0,48$, grande).

A conclusão é que, para a maioria das queries, **GraphQL não é mais rápido que REST** na API do GitHub. O overhead de processamento da query no lado do servidor tende a anular os ganhos potenciais de uma resposta menor.

4.2. RQ2 — Tamanho da Resposta

Os resultados são contundentes: **GraphQL retorna respostas significativamente menores que REST** em todos os cenários ($p \approx 0,000$ para todas as queries, Cliff's $\tilde{D} = +1,0$).

As reduções mais expressivas ocorrem nas queries de listagem:

- **Q4 (issues):** REST retorna 141.217 bytes vs 13.064 bytes do GraphQL — **redução de 90,7%**
- **Q5 (pull requests):** REST retorna 514.432 bytes vs 15.183 bytes do GraphQL — **redução de 97,0%**
- **Q3 (repositories):** REST retorna 41.788 bytes vs 13.327 bytes do GraphQL — **redução de 68,1%**
- **Q2 (repository):** REST retorna 5.191 bytes vs 633 bytes — **redução de 87,8%**
- **Q1 (user):** REST retorna 1.232 bytes vs 211 bytes — **redução de 82,9%**

Isso confirma a vantagem teórica do GraphQL: ao solicitar apenas os campos necessários, o *overfetching* é eliminado, resultando em payloads dramaticamente menores.

4.3. Análise por Query

O padrão observado revela um trade-off claro:

- **Queries simples (Q1, Q2):** A diferença de tempo é pequena ou inexistente, mas GraphQL já oferece redução significativa no tamanho.
- **Queries de listagem (Q3–Q5):** A redução de tamanho é massiva (68–97%), mas o GraphQL tende a ser mais lento em tempo de resposta, possivelmente devido ao overhead de resolução dos grafos no servidor.
- **Exceção Q4 (issues):** GraphQL foi mais rápido e menor, sugerindo que para certos tipos de recursos, o processamento server-side do GraphQL é otimizado.

4.4. Limitações

- **Variabilidade de rede:** As medições de latência incluem o tempo de rede, que pode variar com o horário e o estado da conexão do ISP.
- **Caching:** Apesar do header **Cache-Control: no-cache**, o GitHub pode empregar mecanismos internos de cache que não são influenciados por este header.
- **Escopo limitado:** Apenas 5 tipos de consulta foram testados, todas de leitura (*read-only*). Operações de mutação não foram avaliadas.
- **Ponto único:** O experimento foi conduzido de um único ponto geográfico, o que limita a generalização dos resultados de latência.
- **Equivalência parcial:** Embora as queries GraphQL tenham sido construídas para solicitar os mesmos campos, a estrutura interna das respostas JSON difere (nesting, metadados), o que pode influenciar levemente os tamanhos.

5. Conclusão

RQ1 — Tempo de Resposta

H₀₁ é rejeitada ($p = 0,000001$): existe diferença estatisticamente significativa no tempo de resposta entre GraphQL e REST. Entretanto, contrariamente à expectativa, **REST tende a ser mais rápido** na maioria das consultas, com efeito de magnitude pequena (Cliff's $\delta = -0,17$). A vantagem do GraphQL em tempo se manifesta apenas em cenários específicos (Q4 — issues).

RQ2 — Tamanho da Resposta

H₀₂ é rejeitada ($p \approx 0,000$): existe diferença altamente significativa no tamanho das respostas. **GraphQL retorna respostas consistentemente menores** em todas as queries, com efeito de magnitude grande (Cliff's $\delta = +0,52$ overall, $\delta = +1,0$ para cada query individual). A redução varia de 68% a 97%.

Implicações Práticas

Para desenvolvedores considerando a migração de REST para GraphQL:

- **Economia de banda é garantida:** GraphQL reduz significativamente o volume de dados transferidos, sendo especialmente benéfico para clientes mobile ou com conexões limitadas.
- **Latência pode não melhorar:** O overhead de processamento no servidor pode anular ganhos de latência, especialmente para consultas de listagem.
- **Decisão contextual:** A escolha entre REST e GraphQL deve considerar o cenário de uso — se a prioridade é minimizar transferência de dados, GraphQL é superior; se a prioridade é minimizar latência, REST pode ser preferível.

Referências

GITHUB. REST API v3 Documentation. Disponível em: <https://docs.github.com/en/rest>.

GITHUB. GraphQL API v4 Documentation. Disponível em: <https://docs.github.com/en/graphql>.

BRITO, G.; MOMBACH, T.; VALENTE, M. T. Migrating to GraphQL: A Practical Assessment. In: IEEE SANER, 2019.

WILCOXON, F. Individual Comparisons by Ranking Methods. Biometrics Bulletin, 1945.

ROMANO, J.; KROMREY, J. D.; CORAGGIO, J.; SKOWRONEK, J. Appropriate Statistics for Ordinal Level Data. In: AERA Annual Meeting, 2006.